FPGA prototyping Report

Student Names
Ahmed Hossam Rafik
Ahmed Haitham Othman
Mohamed Ahmed Mohamed Hussien
Mohamed Adel Abdelrahem
Abdelrhman Khaled Fouad
Youssef Mohamed Mamdouh
Moustafa Saad Dawood

FPGA in The LOOP (FIL)

FIL is a hardware-assisted verification technique where a software simulation running on a host PC (typically in an environment like MATLAB or Simulink) is synchronized with an actual, physical FPGA board. You run the Device Under Test (DUT) on real hardware while the host PC handles the generation of test vectors and the visualization of the results.

How FIL Works:

1. **Generation:** You take your RTL code and use a tool to generate a "wrapper." This wrapper includes your DUT and specialized communication interfaces (usually JTAG, Ethernet, or UART).
2. **Implementation:** The wrapper is synthesized, routed, and generated into a bitstream, which is then programmed onto your physical board.
3. **Co-Simulation:** During the simulation run, the host PC software acts as the ultimate testbench. It generates the input stimuli, sends them over the communication link to the FPGA, waits for the FPGA to process the data in hardware, and then reads the output back to analyze it.

How FIL is implemented in our project:

1. Launch the filWizard from MATLAB and Select the Zybo Z7-20 as the target board.
2. Choose the communication link. Because this system deals with heavy data streams (OFDM and FFT outputs), Ethernet is highly recommended over JTAG for bandwidth reasons
3. Add the new top-level Verilog file and let MATLAB map the I/O ports.
4. MATLAB will generate a Vivado project.
5. Write MATLAB code to generate the OFDM data and define the inputs, Drive the TX Path, Simulate the Analog Channel, Drive the RX Path and Verify the Results

Mapping I/Os on the Zynq-7020 FPGA

There are several options to map our signals on the FPGA

1. Memory-Mapped AXI via the ARM Processor (The "SoC" Approach)

Because our Zynq-7020 chip contains both an FPGA (Programmable Logic) and an ARM Cortex-A9 processor (Processing System), this is the industry-standard methodology for deploying your project.

- **How it works:** Package the Verilog code as a custom AXI4-Lite IP. This maps the input signals to specific memory addresses. Then write a simple C program running on the ARM core. When the C code writes a variable to that memory address, the AXI bus automatically pushes that value directly into the hardware registers.

1. Architectural Overview:

- The system utilizes the Zynq-7000 SoC architecture, dividing tasks between the Processing System (PS) and the Programmable Logic (PL). The ARM Cortex-A9 (PS) is responsible for system initialization, configuring continuous wave parameters, and provisioning the OFDM communication payload. The PL handles the high-speed, real-time Digital Signal Processing (DSP) for both the TX and RX data paths.
- Communication between the PS and PL is facilitated by the Advanced eXtensible Interface (AXI4) standard.

2. Memory Mapping and Control Interface (AXI4-Lite):

To control the system dynamically, the custom PL blocks are packaged with an AXI4-Lite slave interface. This maps the

hardware inputs directly into the ARM processor's memory space, allowing the C software to configure the hardware using standard memory-write operations.

The control variables are mapped into a custom Register File as follows:

- **Register 0 (Base Address + 0x00): FTW_start**
  - Width: 32 bits
  - Function: Defines the initial Frequency Tuning Word for the DDS block.
- **Register 1 (Base Address + 0x04): FTW_step**
  - Width: 32 bits
  - Function: Defines the frequency step size for sweeping or modulation.
- **Register 2 (Base Address + 0x08): cycles**
  - Width: 13 bits (Mapped to the lower 13 bits of the 32-bit register)
  - Function: Dictates the number of operational cycles
- **Register 3 (Base Address + 0x0C): start**
  - Width: 1 bit (Mapped to the Least Significant Bit)
  - Function: The software-triggered execution signal that enables the DSP pipelines.

3. OFDM Payload Provisioning (Data Transfer)

The OFDM block requires 2048 symbols, each 32 bits wide, totaling 8 Kilobytes of data. Because this data must be ready before the TX path begins execution, the ARM processor handles the memory loading.

4. Software Execution Flow (Bare-Metal C Application)

The operational sequence running on the ARM Cortex-A9 (developed in Xilinx SDK or Vitis) follows a strict procedural flow:

1. **System Initialization:** The software initializes the ARM core, configures the AXI bus, and asserts the rst_n signal to clear the PL state.
2. **Data Loading:** The software calculates or retrieves the 2048 OFDM symbols (32 bits each) and writes them sequentially into the PL-side memory (BRAM).
3. **Parameter Configuration:** The software writes the calculated FTW_start, FTW_step, and cycles values to their respective memory-mapped registers.
4. **Execution Trigger:** The software writes a '1' to the start register.
5. **Hardware Hand-off:** Once the start signal is asserted, the ARM processor steps back. The PL hardware takes over, executing the DDS generation, FFT/IFFT processing, and the joint radar-communication pipeline in real-time.
6. **Verification:** The internal comparators check the received payload against the transmitted payload. The results are driven to physical LEDs on the Zybo board to provide immediate,

visual verification of system success without requiring processor intervention.

For this choice the workflow is cleanly split between two main tools: **Vivado** (for designing the hardware architecture) and **Xilinx SDK** (for writing the C software).

-Phase 1: Custom Peripheral Encapsulation (Interface Definition)

To establish a bridge between the Processing System (ARM Cortex-A9) and the Programmable Logic (FPGA Fabric), the custom digital signal processing modules (DDS, TX/RX paths) must be encapsulated within a standardized communication protocol.

- **Action:** The control signals (FTW_start, FTW_step, cycles, start) are mapped to a custom Register File using the AXI4-Lite interface standard. This abstraction converts the raw hardware input ports into explicitly addressable memory locations, allowing the processor to configure the hardware pipeline using standard memory-write operations.

-Phase 2: System-on-Chip (SoC) Topography Construction

With the custom IP packaged, the overarching system architecture is constructed using a block-level integration environment. This phase defines the data routing, clocking domains, and shared memory structures.

- **Action:** The Processing System (PS) is instantiated and configured to match the physical board parameters, supplying the base clock and reset lines to the fabric. A Dual-Port Block

RAM (BRAM) is provisioned to act as the shared memory buffer for the 2048 OFDM symbols. Finally, an automated interconnection network (AXI Interconnect) is synthesized to correctly route the memory-mapped addresses between the ARM master core and the custom hardware slaves.

-Phase 3: Hardware Synthesis and Implementation

This phase translates the high-level block architecture and Register Transfer Level (RTL) code into physical silicon routing.

- **Action:** The abstract system design is wrapped into a top-level hardware description file. Physical constraints are applied to map the digital outputs to the correct physical pins on the Zybo board (e.g., LEDs and DAC interfaces). The design undergoes synthesis and place-and-route optimization, culminating in the generation of a binary bitstream file used to program the physical logic gates of the FPGA.

-Phase 4: Software Stack and Custom Driver Development Once the hardware architecture is finalized and exported, the focus shifts to the software domain to develop the control orchestration.

- **Action:** A custom Board Support Package (BSP) is generated, inherently linking the C software environment to the specific physical addresses defined in Phase 2. Custom bare-metal C drivers are then developed to execute the operational flow: initializing the system, populating the BRAM with the

calculated OFDM payload, configuring the DDS parameters via the AXI4-Lite registers, and ultimately asserting the hardware execution flag to hand off real-time processing to the FPGA fabric.

Scenario of how the ARM CORTEX Processor will drive the inputs to our system

Assuming that C driver of the processor is as follows:

```
Xil_Out32(0x43C00000, 0x00001111); // Write FTW_start
Xil_Out32(0x43C00004, 0x0000000A); // Write FTW_step
Xil_Out32(0x43C00008, 0x00000400); // Write cycles
Xil_Out32(0x43C0000C, 0x00000001); // Assert start bit
```

Transaction 1: Loading FTW_start

- **T_0:** The ARM asserts AWVALID=1 and AWADDR=0x43C00000. The AXI wrapper replies with AWREADY=1. (Address Handshake)
- **T_1:** The ARM asserts WVALID=1 and WDATA=0x00001111. The AXI wrapper replies with WREADY=1. **Your FTW_start wire updates to its final value.**
- **T_2:** The AXI wrapper asserts BVALID=1 (OKAY). The ARM accepts it.

Software Gap 1 (ARM fetches the next instruction)

- **T_3 to T_16:** The AXI bus is idle. Your PHASE_ACC is holding FTW_start, but because start is still 0, it does nothing.

Transaction 2: Loading FTW_step

- **T17:** ARM asserts AWVALID=1, AWADDR=0x43C00004. Handshake succeeds.
- **T_18:** ARM asserts WVALID=1, WDATA=0x0000000A. Handshake succeeds. **Your FTW_step wire updates to its final value.**
- **T_19:** Response handshake succeeds.

Software Gap 2

- **T_20 to T_33:** Bus is idle. The ARM is processing the third instruction.

Transaction 3: Loading cycles

- **T_34:** ARM asserts AWVALID=1, AWADDR=0x43C00008. Handshake succeeds.
- **T_35:** ARM asserts WVALID=1, WDATA=0x00000400. Handshake succeeds. **Your cycles wire updates to 1024.**
- **T_36:** Response handshake succeeds.

Software Gap 3 (The final pause before execution)

- **T_37 to T_50:** All parameters are fully loaded and stable on the input pins of your dds_top module. The hardware is armed.

Transaction 4: The Trigger (start)

- **T_51:** ARM asserts AWVALID=1, AWADDR=0x43C0000C. Handshake succeeds.
- **T_52:** ARM asserts WVALID=1, WDATA=0x00000001. Handshake succeeds. **Your start wire goes high.**
- **T_53:** Response handshake succeeds. The ARM is now completely finished configuring the DDS.

The Handoff to Your Logic

Because your start wire went high at T_52, your Verilog code immediately takes over on the very next rising edge for the system to start its functionality

How are the OFDM symbols stored to be driven to the system?

Method 1: The DRAM Approach (Matches your diagram)

In this scenario, the OFDM symbols are stored in the DDR RAM connected to the ARM processor.

How it is stored: You calculate your OFDM symbols (e.g., using MATLAB) and export them as a large array of numbers. In your Vitis/SDK C code running on the ARM

We add an AXI Direct Memory Access (DMA) IP block to your Vivado block design.

1. Setup: The ARM uses AXI4-Lite to configure the DMA, telling it the starting address of ofdm_symbols in the DRAM and the total number of bytes to send.
2. Execution: The ARM triggers the DMA to start.

3. The Handoff: The DMA bypasses the processor entirely. It rapidly pulls the OFDM data from the DRAM and blasts it into your Programmable Logic (PL) using an AXI4-Stream interface. The Verilog code will need to accept this streaming data (using standard tdata, tvalid, and tready signals) and feed it into TX multiplexer.

Method 2: The Hardware ROM Approach

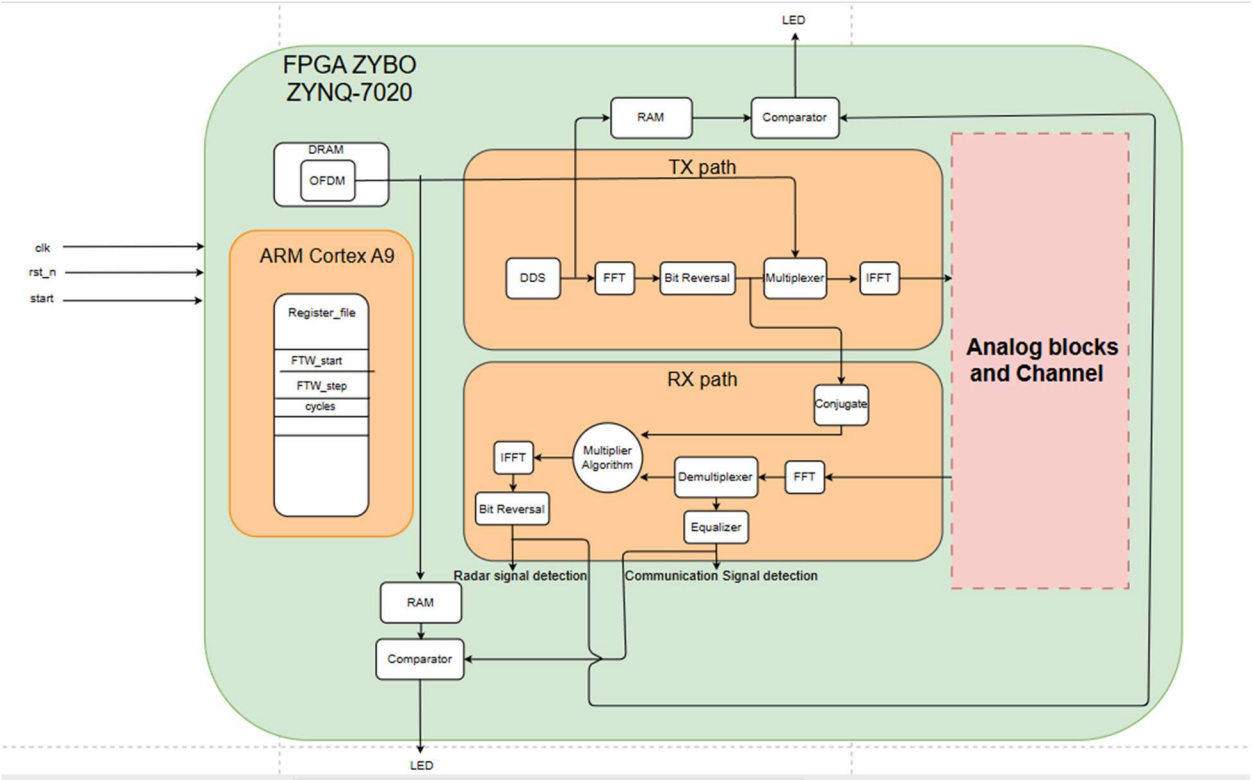
If you want to bake the OFDM symbols permanently into the hardware so the ARM doesn't have to manage memory transfers, we implement it in the PL fabric using Block RAM (BRAM).

How it is stored: we format our pre-calculated OFDM symbols into a .mem or .coe file. In Vivado, then instantiate a Block Memory Generator IP, configure it as a Single-Port ROM, and point it to the initialization file.

How it is driven (Custom RTL Controller):

1. Setup: The ARM writes a start bit over an AXI4-Lite interface.
2. Execution: Your custom Verilog logic detects the start bit and enables a counter.
3. The Handoff: On every clock cycle, the counter increments the address line of the BRAM. The BRAM spits out the next OFDM symbol directly into your TX multiplexer. The ARM does zero work during this process.

Block Diagram



Option 2

1. Implementing the Comms Comparator (Bottom LED)

This comparator verifies the communication link by ensuring the data recovered by the Equalizer matches the original data stored in the Reference RAM.

Dual-Routing Strategy: When the start signal is high, the Multiplexer pulls OFDM data from the ROM, sending it simultaneously to the TX IFFT and the Reference RAM.

- **Latency Management:** The RAM acts as a synchronous queue, holding the transmitted data until the signal has traveled through the full pipeline and analog channel.
- **Comparison Phase:** When the Equalizer asserts a valid signal, the comparator pops the corresponding word from the RAM for a bit-for-bit check.
- **Sticky Fail Logic:** An error register (initialized to 0) captures any mismatch. If a mismatch occurs, it "sticks" to 1, keeping the Bottom LED OFF for the remainder of the frame to indicate a link failure.

2. Implementing the Radar Comparator (Top LED)

This comparator verifies the radar link by checking if the processed return matches the live golden reference signature.

- **Live Reference Generation:** As the DDS generates the chirp, the resulting spectral signature from the TX Bit Reversal block is dual-routed to the TX Multiplexer and the Radar Comparator.
- **In-Phase Comparison:** The hardware comparator waits for a valid signal from the RX path's bit reversal block to perform the check: TX_Bit_Reversal_out == RX_Bit_Reversal_out.
- **Success Indicator:** If the signatures match (within a defined tolerance for channel noise), the Top LED is driven high.

3. Pure Hardware Control & Real-Time Testcases

Your design now features a 2-pin selection system to allow real-time testing of different LFM chirp characteristics without re-synthesizing the FPGA.

A. The Multi-Case Register File

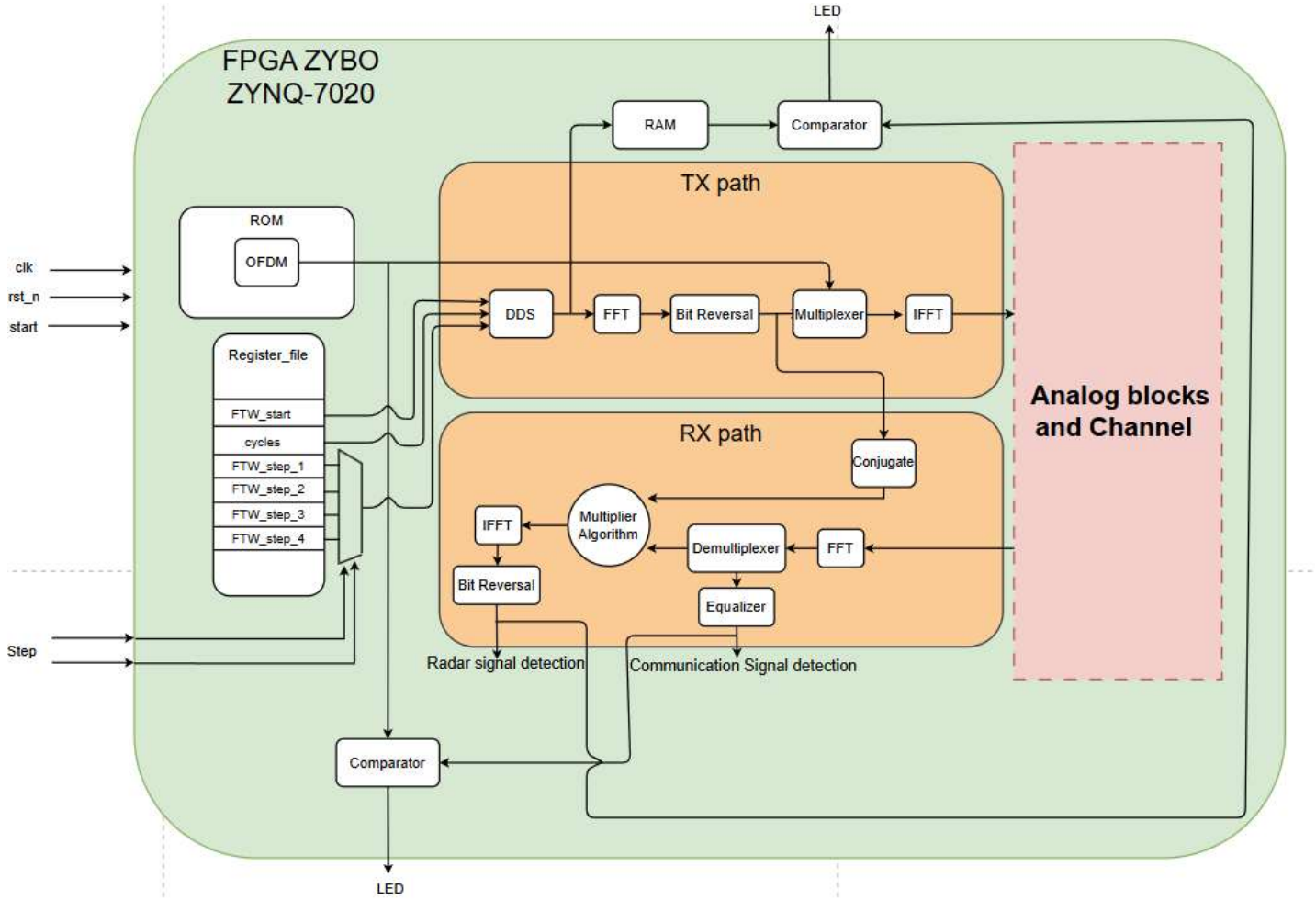
Instead of a single fixed step, the Register_file now holds four unique values for FTW_step:

- **Register_file Content:** Stores FTW_start, cycles, and the array FTW_step_1 through FTW_step_4.
- **Real-Time Selection (Step Pins):** You have 2 external pins wired to the select line of a hardware multiplexer within the Control Plane. This allows you to toggle between four distinct radar resolution/bandwidth scenarios on the fly by changing the input on these pins.

B. Data & Orchestration

- **ROM Instantiation:** The golden OFDM payload is "baked" into the FPGA bitstream using hardcoded ROM.
- **Hardware start:** start trigger (e.g., a push-button), ensuring the DDS and ROM stream start at the exact same clock cycle.

Summary of the Test Flow

- **When the physical start signal is asserted:**
    - **LED Reset:** Both LEDs turn OFF as the internal error registers are cleared.
    - **DDS & Step Selection:** The DDS immediately pulls FTW_start and cycles from the Register File. Simultaneously, it pulls the specific FTW_step (1-4) currently selected by your 2-pin physical interface.
    - **Parallel Data Launch:** The Multiplexer begins pulling pre-loaded OFDM data from the ROM, streaming it into the TX path and the Reference RAM.
    - **Signal Propagation:** The system processes the radar chirp and communication data through the analog blocks and channels.
    - **Simultaneous Verification:**
    - **Communication:** If RAM_out == Equalizer_out, the Top LED turns ON.
    - **Radar:** If the processed return matches the reference (TX_Bit_Reversal_out == RX_Bit_Reversal_out), the Bottom LED turns ON

Block Diagram